



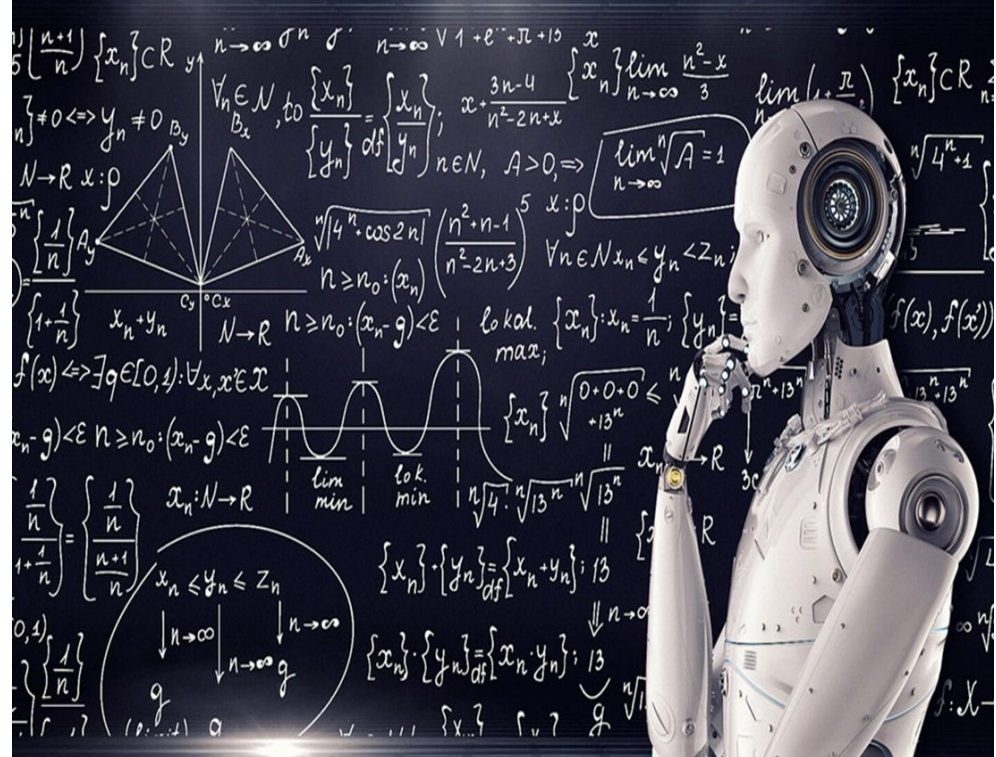
Technische Hochschule
Ingolstadt

Developing an Autonomous Driving agent in Metadrive using PPO

Content



- Introduction
- Metadrive Environment
- Methodology
- Results & Performance
- Discussion
- Open Research Challenges
- Future Scope
- Conclusions
- Learning
- References



Introduction



- **Introduction:**

- Designed and implemented a Proximal Policy Optimization (PPO) framework for training an autonomous driving agent in the MetaDrive simulation environment.
- The agent's task is to navigate traffic, intersections, highways, and sharp curves while ensuring safety and following traffic rules.
- Developed a reward mechanism to promote safe and efficient driving behaviors while penalizing unsafe actions, such as collisions, going off-road, and violating traffic rules.
- Used PyTorch framework and a clipping mechanism to ensure stable policy updates during training.
- Demonstrated the agent's ability to navigate complex driving environments effectively, showcasing its robustness and adaptability for real-world autonomous driving applications.
- This presentation highlights the objectives, methodology, and key outcomes of this project, emphasizing the role of PPO and Metadrive.

MetaDrive Simulation:



❖ What is MetaDrive?

- MetaDrive is a Simulation Environment to train and test Autonomous driving agents that can generate realistic road networks and traffic scenarios.

➤ Features of MetaDrive:

- Procedural Road Generation
- Dynamic Traffic
- Realistic Physics
- Customizability

Episode:0
Timestep:0
Traffic Density:0.3
Speed:N/A
Collision Vehicle:False
Out of Road:False
Distance Completion:0.00%

❖ Why MetaDrive is Ideal for PPO?

- Continuous Action Space
- High-dimensional Observations
- Realistic Driving Challenges
- Clipped Optimization



MetaDrive with PPO:



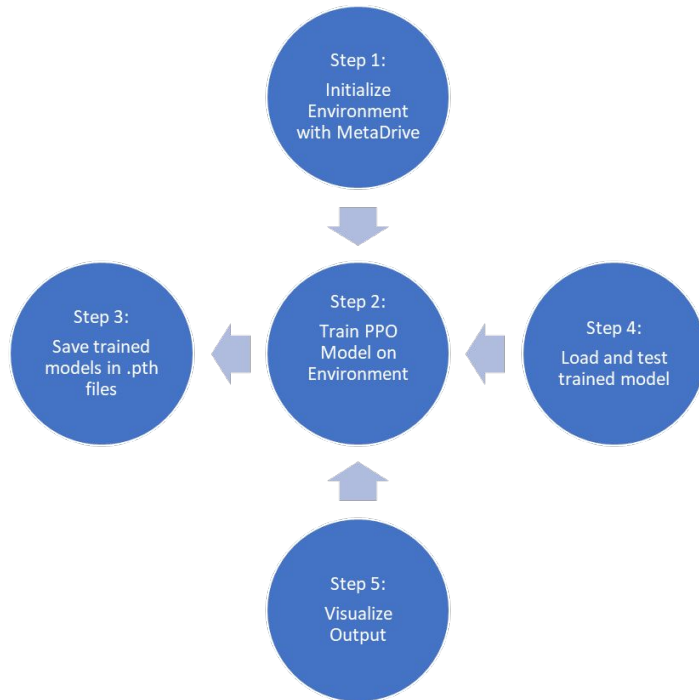
1. Initialization

2. Step-by-Step Interaction:

- The agent receives a 259-dimensional observation (LiDAR, speed, position).
- The PPO Actor predicts a continuous action.
- The action is applied in MetaDrive, and the environment updates:
 - New Observation.
 - Reward: Progress (+), collision (-), off-road penalty (-).
 - Done Flag: Episode ends if collision or task is completed.
- PPO uses the collected data to update the Actor and Critic.

3. Training:

- Actor
- Critic
- Policy Update





How MetaDrive is better than Other Simulators?



Feature	MetaDrive	CARLA	AirSim
Procedural Road Design	Yes	No(Fixed Maps)	No(Fixed Maps)
Continuous Action Space	Yes	Limited	Yes
Lightweight	Yes(Efficient with CPU)	No(GPU Intensive)	No(GPU Intensive)
Traffic Simulation	Yes	Yes	Limited
Reinforcement Learning	Optimized	Requires Customization	Requires Customization

MetaDrive Environment



4.2. MetaDrive Features Used in the Project

- **Observation Space (259 – Dimensional Vector):**
 - A 259-dimensional vector including LiDAR data, vehicle speed, and positional information.

Feature Type	Count	Description
LiDAR Data	240	Distance readings from LiDAR sensors for obstacle detection.
Lane Information	7	Details of vehicle position and orientation relative to the lane.
Vehicle State	8	Speed, heading, and other vehicle dynamics.
Positional Information	4	Distance and direction to the next waypoint or goal.
Total	259	

- **Action Space:**

The action space tells the vehicle what actions it can take. It has two main controls:

 - **Steering:** A value between -1 (full left turn) and 1 (full right turn).
 - **Throttle:** A value between 0 (no acceleration) and 1 (full acceleration).

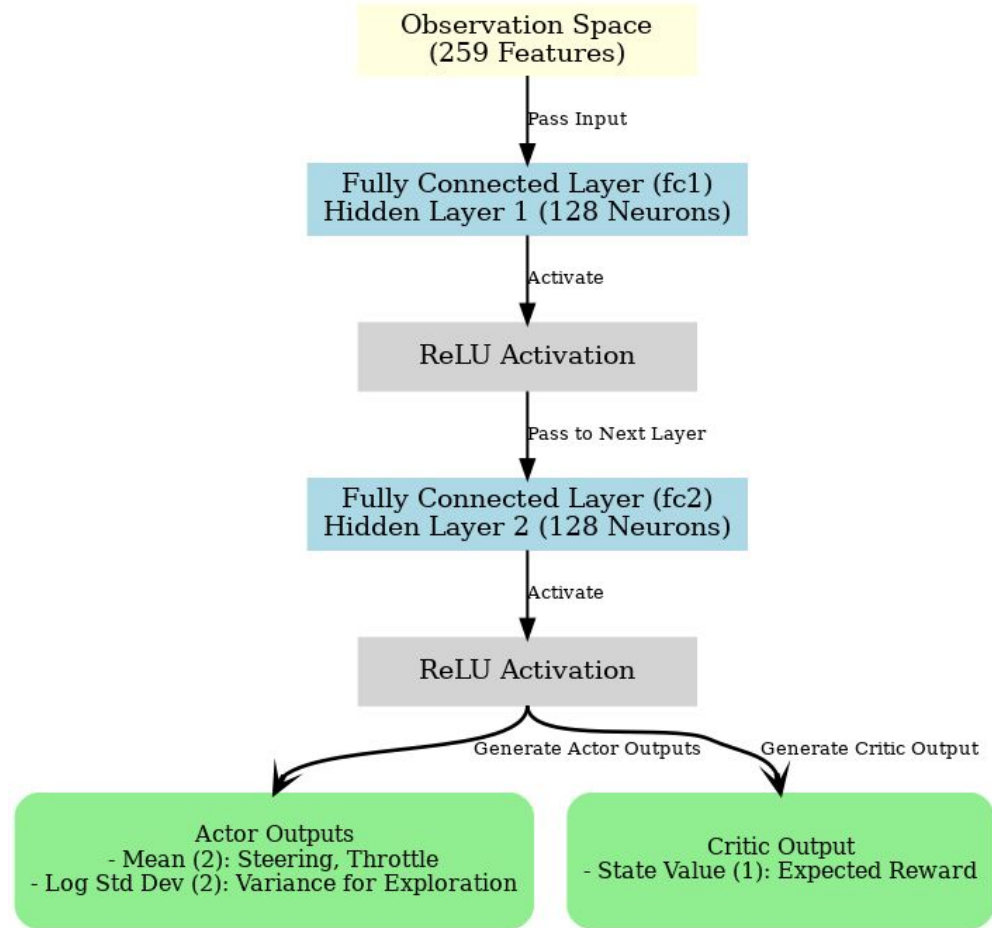


Fig. Neural Network Design for Processing MetaDrive Features

MetaDrive Environment



- **Reward Structure:**

- Rewards are integral to training the agents in MetaDrive, encouraging desirable behaviors through positive reinforcement and providing negative reward for undesirable behaviors.

$$R = c_1 R_{driving} + c_2 R_{speed} + R_{termination}$$

```
"use_lateral_reward": True, # Reward for maintaining lateral position
"success_reward": 25.0, # Moderate success reward for balanced learning
"driving_reward": 2.0, # Increased driving reward to encourage smooth driving
"out_of_road_penalty": -8.0, # High penalty for going out of road
"crash_vehicle_penalty": -15.0, # Large penalty for crashing into vehicles
"crash_object_penalty": -8.0, # Moderate penalty for crashing into objects
```

- **Cost Function:**

- we also provide a default cost function to measure safety during driving.
- The cost function will be placed in the returned information dict as `info["cost"]` after `env.step` function.

MetaDrive Environment



- **Termination Conditions:**

- The criteria under which an episode concludes.

- **Truncation Conditions:**

- Truncation conditions specify when an episode should be artificially cut short before the natural termination triggers.

```
"crash_object_done": True, # Ends episode when crashes into object
"crash_vehicle_done": True, # Ends episode when vehicle crashes
"out of route done": True, # Ends episode when off route
```

- **Step Information:**

- The step information dict `info` contains rich information about current state of the environment and the target vehicle. The step info is collected from various sources such as the engine, reward function, termination function, traffic manager, agent manager and so on.

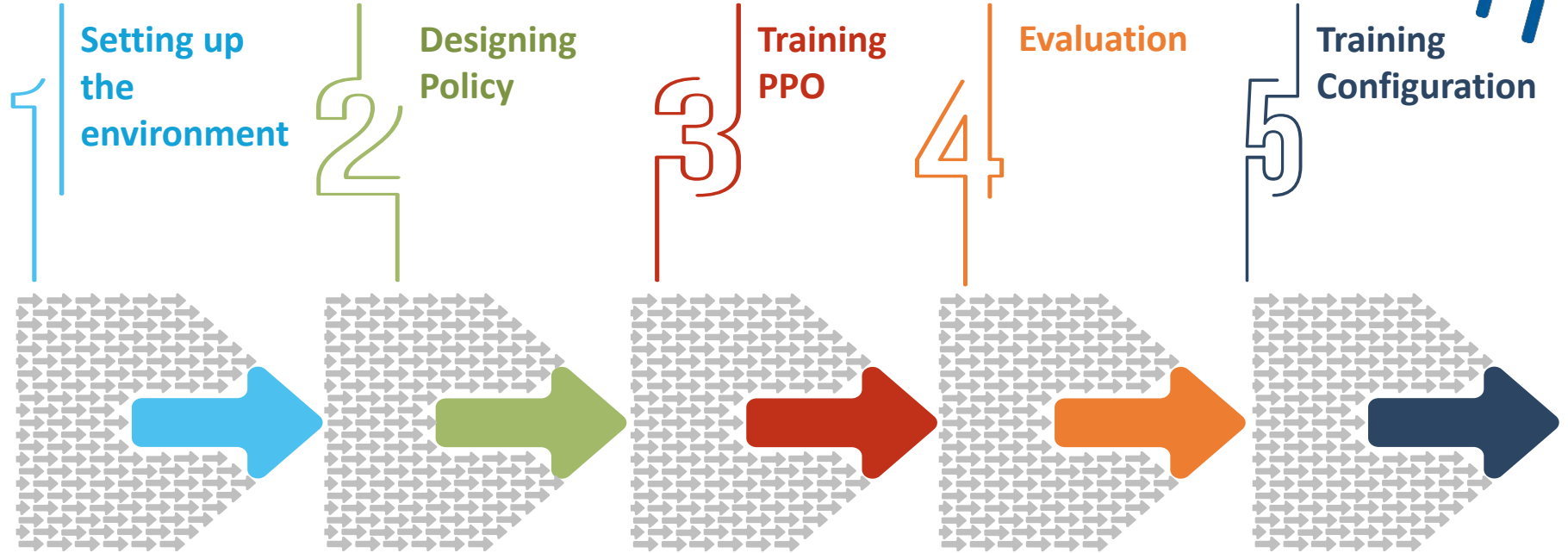
```
[INFO] Episode ended! Scenario Index: 0 Reason: out_of_road.
Epoch 2/2, Avg Reward: 48.49, Actor Loss: -0.42, Critic Loss: 0.59
- Info: {'overtake_vehicle_num': 0, 'velocity': 0.002456969108998885, 'steering': 0.11954735964536667, 'acceleration': -0.21131791174411774, 'step_energy': 0.0004133177032628268,
'episode_energy': 1.3642467264179108, 'policy': 'MetaPPOPolicy', 'navigation_command': 'forward', 'navigation_forward': True, 'navigation_left': False, 'navigation_right': False, 'action': [0.11954736, -0.21131791], 'raw_action': (0.11954736, -0.21131791), 'crash_vehicle': False, 'crash_object': False, 'crash_building': False, 'crash_human': False, 'crash_sidewalk': True, 'out_of_road': True, 'arrive_dest': False, 'max_step': False, 'env_seed': 0, 'crash': True, 'step_reward': 1.105636099049498e-05, 'route_completion': 0.31477335946602875, 'cost': 1.0, 'episode_reward': 48.4895895677208, 'episode_length': 2565}
```



4.3. Challenges in MetaDrive

- Complexity of Simulated Environments- Realistic, Diverse
- Scalability and Performance- Computational resources and scalability
- Integration with Real Hardware - sensor simulation, Hardware compatibility
- Behavioral Modeling- Agent behaviour, Interaction Dynamics
- Learning Algorithms- Convergence and stability
- Safety and Ethical Considerations- Safety validation, Ethical concerns
- Regulatory and Standardization Issues- Compliance with law

Methodology



Methodology

Setting up the environment

Environment Setup:

- Created the MetaDrive environment using the `create_env_with_ppo_policy` function.
- Processed raw observations into normalized inputs for the model.

```
def create_env_with_ppo_policy(actor_model, device):
    class MetaPPOPolicy(PPOPolicy):
        def __init__(self, obj, seed):
            #...
            pass
        env_config = {
            "use_render": False,
            "traffic_density": 0.2,
            #...
        }
        env = MetaDriveEnv(config=env_config)
    def observation_processing(obs):
        #...
        pass
    env.process_observation = observation_processing
    return env
```

```
def _preprocess_observation(self, observation):
    processed_obs = []
    lidar_data = np.zeros(240)
    processed_obs.extend(lidar_data)
    if "position" in observation:
        processed_obs.extend(observation["position"])
    else:
        raise KeyError("Position key is missing in the observation.")
    if "velocity" in observation:
        processed_obs.extend(observation["velocity"])
    else:
        raise KeyError("Velocity key is missing in the observation.")
    ...
    return np.array(processed_obs, dtype=np.float32)
```



Methodology



Designing Policy

Policy Network:

- Generates action distributions based on the input state.
- Architecture:
 - Two fully connected layers.
 - Outputs:
 - Mean (μ) and standard deviation (std) for Gaussian actions.

```
class Actor(nn.Module):
    def __init__(self, input_dim=259, action_dim=2, hidden_dim=128):
        super(Actor, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.mu_head = nn.Linear(hidden_dim, action_dim)
        self.log_std_head = nn.Linear(hidden_dim, action_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        mu = torch.tanh(self.mu_head(x))
        log_std = torch.clamp(self.log_std_head(x), -2, 2)
        std = log_std.exp()
        return Normal(mu, std)
```

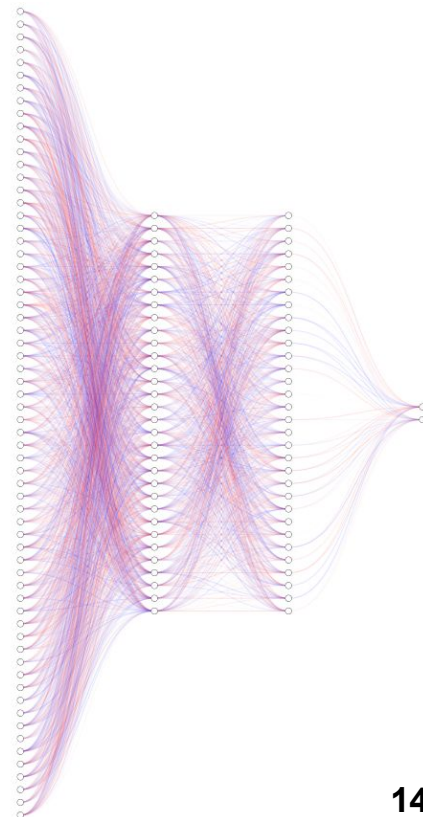


Fig. Policy Network.[LeNail(2019,)NN SVG]

Methodology



Designing Policy

act:

- Selects an action based on the current observation using the policy network.

```
def act(self, agent_id):
    observation = self.control_object.get_state()

    if isinstance(observation, dict):
        observation = self._preprocess_observation(observation)

    obs_tensor = torch.tensor(observation, dtype=torch.float32, device=self.device).unsqueeze(0)

    with torch.no_grad():
        dist = self.actor_model(obs_tensor)
        action = dist.sample().squeeze().cpu().numpy()

    action = [clip(action[i], -1.0, 1.0) for i in range(len(action))]

    self.action_info["action"] = action
    return action
```

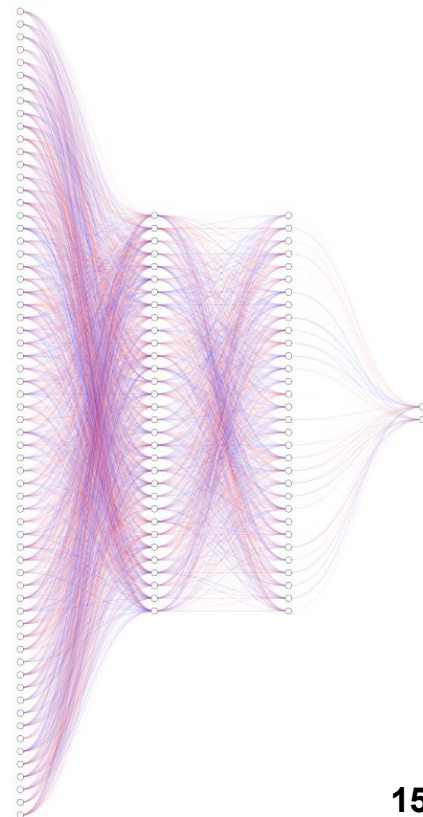


Fig. Policy Network.[LeNail(2019,)NN SVG]

Methodology



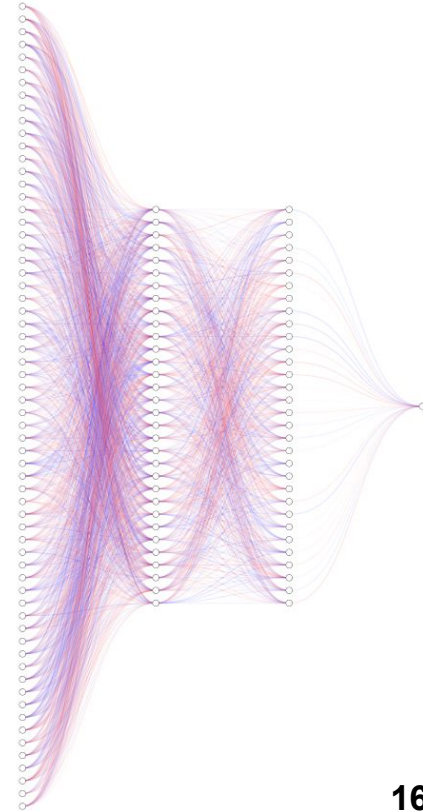
Designing Policy

Value Network:

- Predicts the value of a state (expected cumulative reward).
- Architecture:
 - Two fully connected layers.
 - Outputs:
 - A single scalar value.

```
class Critic(nn.Module):
    def __init__(self, input_dim=259, hidden_dim=128):
        super(Critic, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.value_head = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        value = self.value_head(x)
        return value
```



Methodology



Training PPO

Data Collection:

- Collect trajectories (observations, actions, rewards) from multiple episodes.

Calculations:

- Compute Rewards-to-Go (RTG).
- Calculate Advantages using Generalized Advantage Estimation (GAE).

Training:

- Update the Actor network using PPO loss with clipping.
- Update the Critic network using Mean Squared Error (MSE) loss.

```
def train_ppo(actor, critic, actor_optimizer, critic_optimizer, obs_batch, act_batch, adv_batch, rtg_batch, config):
    obs_batch = torch.from_numpy(np.array(obs_batch, dtype=np.float32)).to(actor.fc1.weight.device)
    act_batch = torch.from_numpy(np.array(act_batch, dtype=np.float32)).to(actor.fc1.weight.device)
    adv_batch = torch.from_numpy(np.array(adv_batch, dtype=np.float32)).to(actor.fc1.weight.device)
    rtg_batch = torch.from_numpy(np.array(rtg_batch, dtype=np.float32)).to(actor.fc1.weight.device)

    old_log_probs = actor(obs_batch).log_prob(act_batch).sum(dim=-1).detach()
    for _ in range(config.ppo_grad_descent_steps):
        actor_loss, new_log_probs = compute_ppo_loss(actor, obs_batch, act_batch, adv_batch, old_log_probs, config.ppo_eps)
        actor_optimizer.zero_grad()
        actor_loss.backward()
        actor_optimizer.step()

    value_preds = critic(obs_batch).squeeze()
    critic_loss = F.mse_loss(value_preds, rtg_batch)
    critic_optimizer.zero_grad()
    critic_loss.backward()
    critic_optimizer.step()

    return actor_loss.item(), critic_loss.item()
```

```
for epoch in range(epochs):
    for _ in range(epochs_per_batch):
        obs = env.reset()
        while not done:
            action = actor(obs).sample()
            next_obs, reward, done, _ = env.step(action)
            trajectories.append((obs, action, reward))

            # Compute rewards-to-go (RTG)
            rtg = compute_rtg(rewards, gamma)
            advantages = rtg - critic(obs)

            # Update Actor and Critic
            actor_loss, critic_loss = train_ppo(actor,
            critic, actor_optimizer, critic_optimizer,
            obs, actions, advantages, rtg)
```

```
def compute_ppo_loss(actor, obs, actions, advantages, old_log_probs, clip_eps):
    dist = actor(obs)
    log_probs = dist.log_prob(actions).sum(dim=-1)
    ratio = torch.exp(log_probs - old_log_probs)
    clipped_ratio = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps)
    loss = -torch.min(ratio * advantages, clipped_ratio * advantages).mean()
    return loss, log_probs
```

Methodology



Evaluation

- **Performance Evaluation:**
 - Evaluate the trained agent in unseen MetaDrive scenarios.

```
for episode in range(num_episodes):
    obs = env.reset()
    total_reward = 0
    while not done:
        action = actor(obs).sample()
        obs, reward, done, _ = env.step(action)
        total_reward += reward
    print(f"Episode Reward: {total_reward}")
```





Training Configuration

- Learning rate for Actor: (1e-4), (1e-3), (1e-5). The actor is responsible for updating the policy, which directly affects the actions taken. A smaller learning rate ensures gradual updates, preventing drastic policy changes that could destabilize training.
- Learning rate for Critic: (1e-3), (1e-5), (1e-4). The critic estimates the value function $V(s)$, which guides the actor's updates by providing feedback on expected rewards. The separate learning rates for actor and critic allow fine-tuned optimization for policy and value approximation.
- Discount factor (γ): γ determines the importance of future rewards relative to immediate rewards. $\gamma=0.99$ implies that the agent considers long-term rewards while still prioritizing nearer-term outcomes.
- PPO clipping parameter (ϵ): PPO restricts policy updates to a small range to prevent drastic changes, ensuring stable training. $\epsilon=0.2$ is a commonly used value in PPO implementations.
- Gradient descent steps per update: This controls how many optimization steps are taken for each batch of data. 10 steps ensure sufficient updates are made per batch to converge toward an optimal policy and value function.

These hyperparameters are carefully chosen to balance stability, learning speed, and overall performance. Their effectiveness lies in preventing large updates, ensuring meaningful feedback, and optimizing the policy incrementally for robust and scalable training.

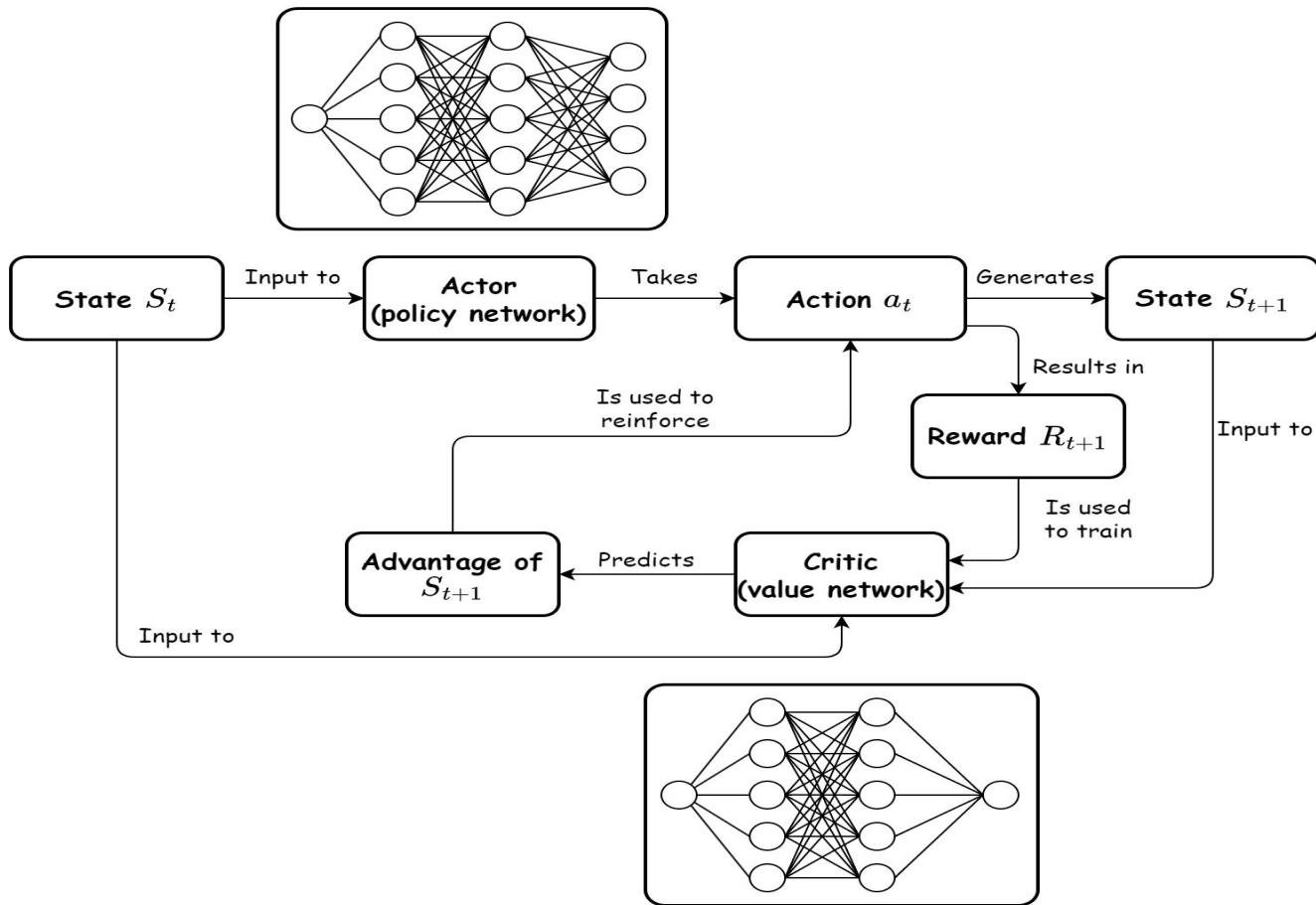
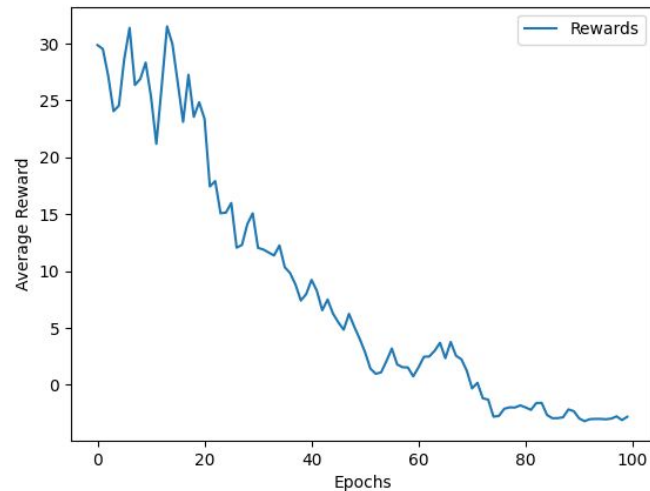
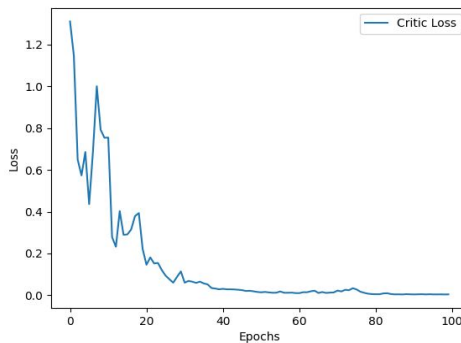
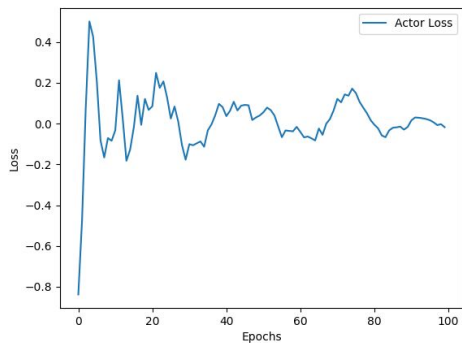


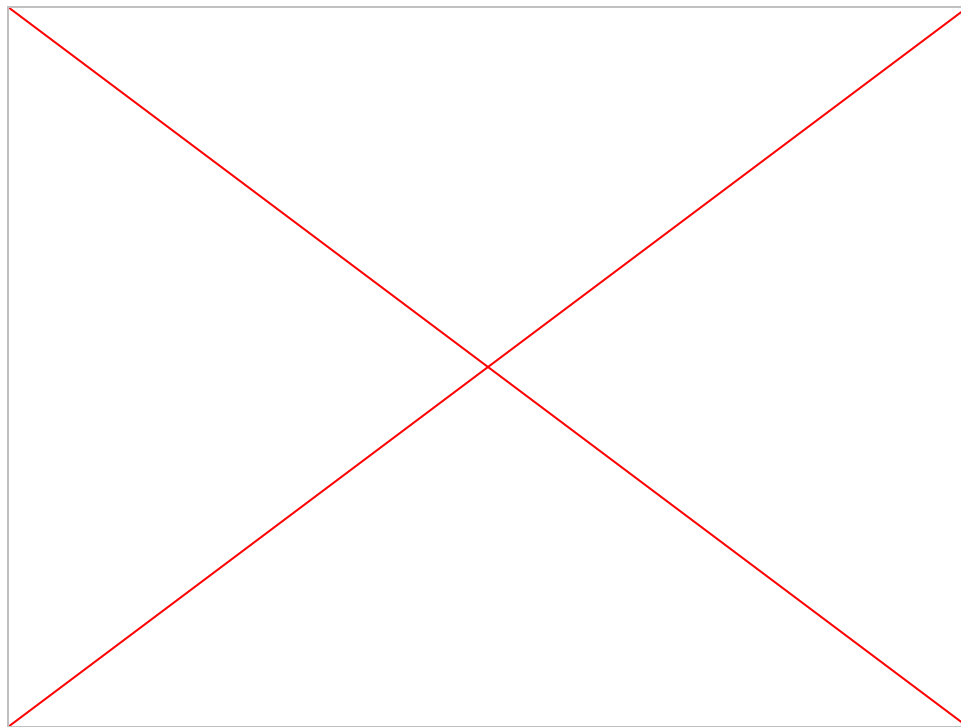
Fig. General overview of actor-critic model in PPO

Results & Performance



```
✓ hyperparams = {  
    "device": torch.device("cuda" if torch.cuda.is_available() else "cpu"),  
    "obs_dim": 259,  
    "action_dim": 2,  
    "actor_lr": 1e-2,  
    "critic_lr": 1e-3,  
    "ppo_eps": 0.2,  
    "ppo_grad_steps": 10,  
    "epochs": 100,  
    "episodes_per_batch": 16,  
    "gamma": 0.98,  
}
```

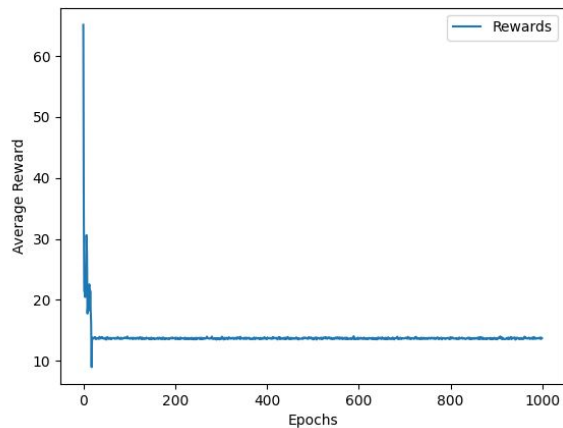
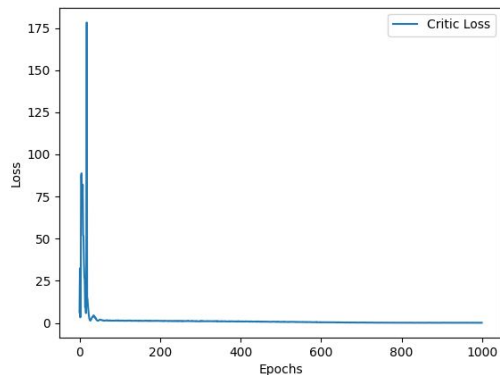
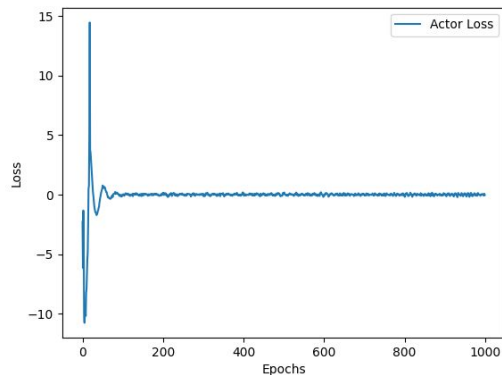




Results & Performance



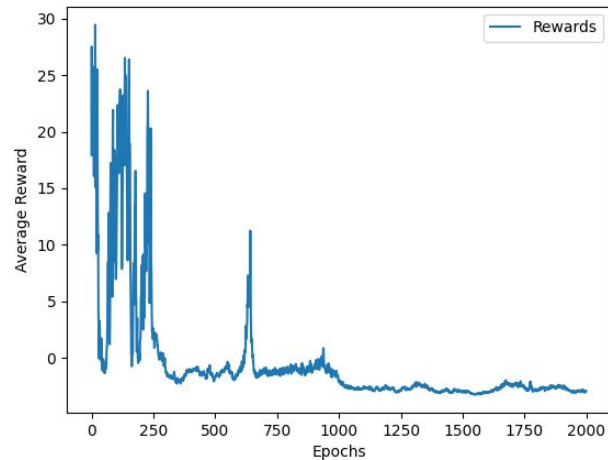
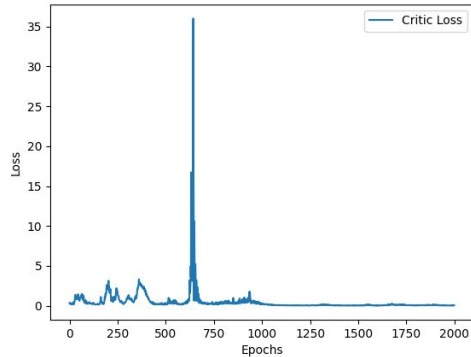
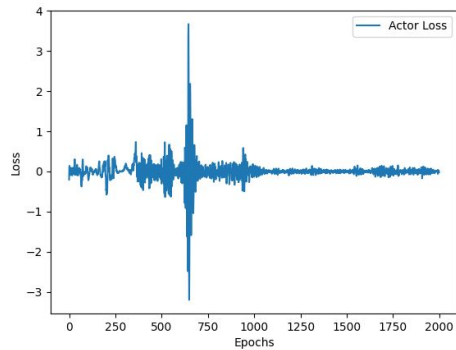
```
hyperparams = {  
    "device": torch.device("cuda" if torch.cuda.is_available() else "cpu"),  
    "obs_dim": 259,  
    "action_dim": 2,  
    "actor_lr": 1e-4,  
    "critic_lr": 1e-5,  
    "ppo_eps": 0.2,  
    "ppo_grad_steps": 10,  
    "epochs": 1000,  
    "episodes_per_batch": 32,  
    "gamma": 0.99,  
}
```

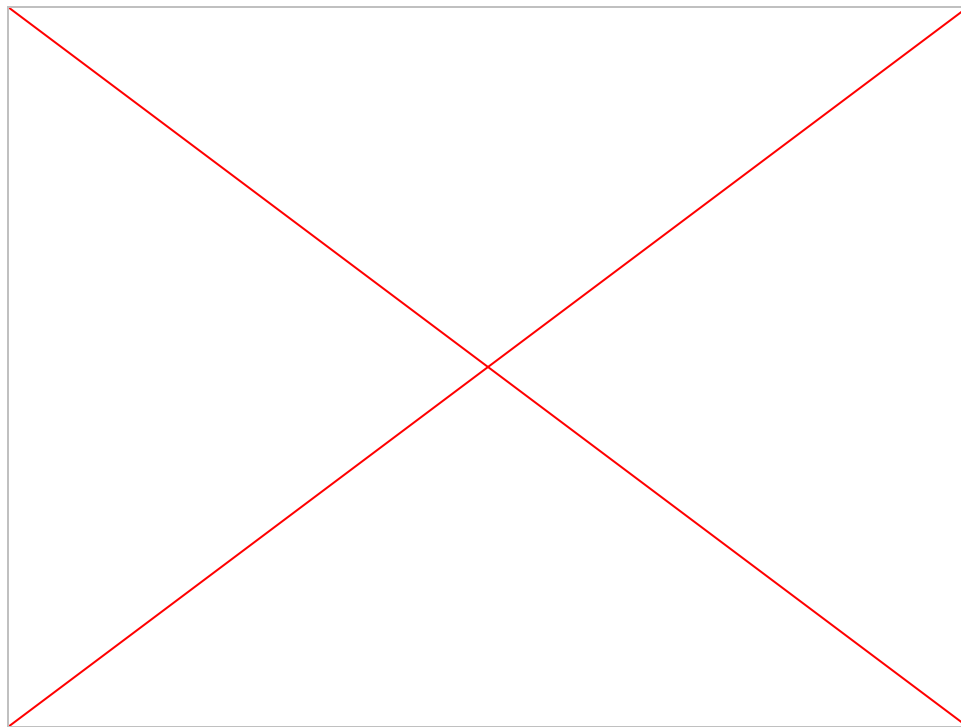


Results & Performance



```
hyperparams = {  
    "device": torch.device("cuda" if torch.cuda.is_available() else "cpu"),  
    "obs_dim": 259,  
    "action_dim": 2,  
    "actor_lr": 1e-4,  
    "critic_lr": 1e-5,  
    "ppo_eps": 0.2,  
    "ppo_grad_steps": 15,  
    "epochs": 2000,  
    "episodes_per_batch": 16,  
    "gamma": 0.99,  
}
```







Further improvements in the project:

- **Potential code errors can be corrected through excessive training and testing of model.**
- **Suboptimal reward calculation can lead to effective learning.**

How can we make it more successful:

- **Explore alternative optimization algorithms for improved convergence.**
- **Enhance the neural network architecture for better feature extraction and learning efficiency.**



Challenges:

1. **Sparse Rewards:**

- Difficulty in obtaining consistent feedback for intermediate actions.

2. **High Dimensional Input/Action Space:**

- Stabilizing training with large state and action spaces.

3. **Environment Variability:**

- Handling dynamic traffic scenarios and diverse road structures.

4. **Long Training Times:**

- Training RL models in simulation required significant computational resources and time.

5. **Reward Function Tuning:**

- Balancing penalties and rewards to ensure learning efficiency and safety.

Open Research Challenges

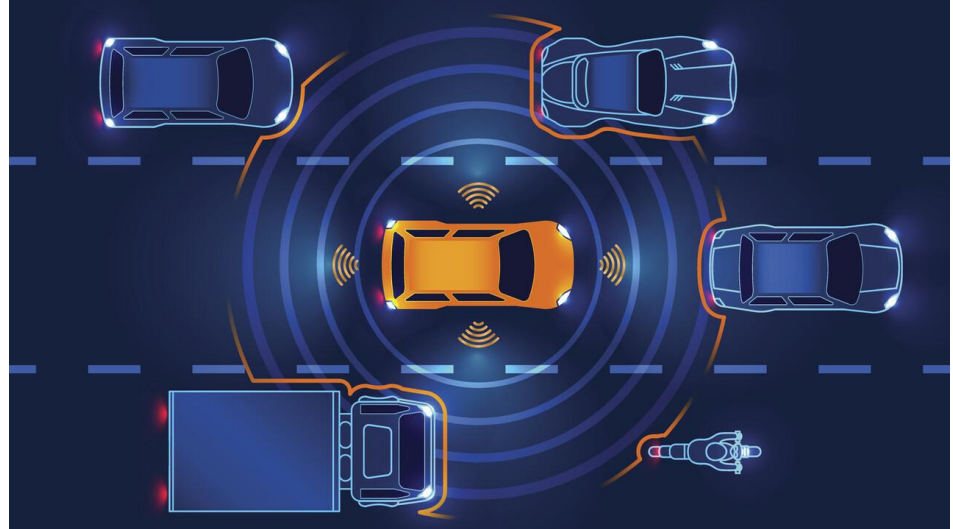


- Bridging the gap between simulated environments (MetaDrive) and real-world applications is a critical challenge. Simulations lack the unpredictability, sensor noise, and diverse conditions of real-world driving.
- Ensuring that policies trained in simulation generalize well to diverse real-world conditions without extensive retraining.
- Managing computational inefficiency and overfitting in environments with high-dimensional input data, such as LiDAR point clouds or video feeds.
- Training agents that can adapt to sudden changes in traffic, road conditions, or weather.
- Designing agents capable of making decisions in morally ambiguous situations (e.g., trolley problem scenarios).
- Addressing issues where agents forget previously learned tasks when training for new ones.

Future Scope



- Domain Adaptation
- Collaboration with Other Vehicles
- Continuous Learning
- More Realistic Scenarios
- Better Use of Road Information
- Balanced Rewards
- Use in Semi-Autonomous Cars



Conclusion



- The PPO algorithm proves to be a robust and efficient RL method for training autonomous driving agents in dynamic and complex environments.
- The agent successfully navigates diverse scenarios in the MetaDrive simulator, such as intersections, curves, and roundabouts, demonstrating adaptability to different road structures.
- The reward mechanism ensures that the agent prioritizes safe and efficient driving, adhering to traffic rules while minimizing collisions and off-road behavior.
- The modular design of the PPO implementation and MetaDrive environment provides a solid foundation for future research, including multi-agent interactions, sensor fusion, and urban driving scenarios.
- The project validates the application of reinforcement learning techniques like PPO for autonomous driving, contributing to the growing body of knowledge in AI and RL.
- It highlights the value of using simulation platforms like MetaDrive for developing, testing, and refining autonomous systems.

Learnings



- The project demonstrates how PPO, a robust and efficient RL algorithm, can be applied to train autonomous driving agents. It balances exploration and exploitation while ensuring stable policy updates using a clipped surrogate objective.
- The project leverages MetaDrive, a versatile driving simulation platform, to train and test agents in varied driving scenarios like intersections, ramps, and roundabouts. This shows the significance of realistic simulations in bridging theoretical models with real-world applications.
- Separate actor and critic networks were designed for policy prediction and value estimation, demonstrating the architecture of actor-critic methods in RL.
- The structured approach to training and testing in a simulated environment suggests the potential for transferring the trained model to real-world autonomous systems.
- The performance metrics, such as average rewards, episode lengths, and success rates, provided clear indicators of the agent's progress. This reinforces the value of systematic evaluation in RL projects.

Reference



- V. Mnih, K. Kavukcuoglu, D. Silver, et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” OpenAI, Jul. 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>.
- V. Mnih, A. P. Badia, M. Mirza, et al., “Asynchronous methods for deep reinforcement learning,” arXiv preprint, Feb. 2016. [Online]. Available: <https://arxiv.org/abs/1602.01783>.
- Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Sample efficient actor-critic with experience replay,” arXiv preprint, Nov. 2016. [Online]. Available: <https://arxiv.org/abs/1611.01224>.
- G. Brockman, V. Cheung, L. Pettersson, et al., “Openai gym,” arXiv preprint, Jun. 2016. [Online]. Available: <https://arxiv.org/abs/1606.01540>.
- MetaDrive Team, Metadrive simulator documentation, Accessed on 2023-01-07, 2023. [Online]. Available: <https://metadrive-simulator.readthedocs.io/en/latest/index.html>.
- A. Kendall, J. Hawke, D. Janz, et al., “Learning to drive in a day,” in International Conference on Robotics and Automation, 2019.
- L. van der Maaten and G. Hinton, “Visualizing data using t-sne,” Journal of Machine Learning Research, 2008.
- B. R. Kiran, I. Sobh, V. Talpaert, et al., “Deep reinforcement learning for autonomous driving: A survey,” IEEE Transactions on Intelligent Transportation Systems, 2021
- https://www.researchgate.net/publication/367165645_Comparative_Study_of_Cooperative_Platoon_Merging_Control_Based_on_Reinforcement_Learning
- LeNail, (2019). NN-SVG: Publication-Ready Neural Network Architecture Schematics. Journal of Open Source Software, 4(33), 747, <https://doi.org/10.21105/joss.00747>